Lab 6: Gaussian Processes - Modeling and Predicting CO2 measurements

The goals of this lab are:

- Practice using Gaussian processes
- Compare the use of different kernels
- Model CO2 measurements in the atmosphere and make predictions on the CO2 levels in the future

In this lab, we will be using data from the the Mauna Loa Observatory, which has been making CO2 measurements of the atmopshere on top of Maunaloa since the late 1950's. It has one of the longest consistent set of measurements of CO2 measurements ever made. It is one of the sources of data used to model climate change. More information on the measurements can be found at:
https://www.esrl.noaa.gov/gmd/ccgg/about/co2_measurements.html

Created by: Tuan Do

Last modified: Tuan Do

In [1]:
```
# imports here
import numpy as np
import pandas as pd
import pylab as plt

import matplotlib
font = {  'size'   : 20}
matplotlib.rc('font', **font)
```

Part 1 Big Picture

Using the CO2 measurements that have been made in the past, we want to predict CO2 measurements in the future as well as the uncertainty in the prediction. We also want to differentiate between seasonal variability from long term trends in the data.

We have two major questions to address in this lab:

1. **How far into the future can we reliably predict the CO2 level of the atmosphere using Gaussian Processes?**
2. **Did the CO2 level rise slow due to the global COVID19 pandemic?**

Part 2 Get the data

The Mauna Loa Observatory makes their data downloadable here: https://www.esrl.noaa.gov/gmd/ccgg/trends/data.html

We've already downloaded the monthly averaged data into your Lab 6 directory. Load it using the following code.

We will use two of the columns for our work: `decimal date` and `average`, which contains the decimal year for the time and the monthly averaged CO2 concentration in parts per million molecules.

In [2]:
```
# load the data
tab = pd.read_csv('co2_mm_mlo.csv',comment='#')
```

Part 3 Explore the data

Question 1

(3 pts)

Check the shape, and head, plot, visualize, and have a look at the data with whatever ways you think will help your task. What's the time span of the dataset? Etc. Comment throughout on observations.

In [3]:
```
tab.head(10)
```

Out[3]:

	year	month	decimal date	average	deseasonalized	ndays	sdev	unc
0	1958	3	1958.2027	315.70	314.43	-1	-9.99	-0.99
1	1958	4	1958.2877	317.45	315.16	-1	-9.99	-0.99
2	1958	5	1958.3699	317.51	314.71	-1	-9.99	-0.99
3	1958	6	1958.4548	317.24	315.14	-1	-9.99	-0.99
4	1958	7	1958.5370	315.86	315.18	-1	-9.99	-0.99
5	1958	8	1958.6219	314.93	316.18	-1	-9.99	-0.99
6	1958	9	1958.7068	313.20	316.08	-1	-9.99	-0.99
7	1958	10	1958.7890	312.43	315.41	-1	-9.99	-0.99
8	1958	11	1958.8740	313.33	315.20	-1	-9.99	-0.99
9	1958	12	1958.9562	314.67	315.43	-1	-9.99	-0.99

In [4]:
```
# code here
print("Shape:",tab.shape)
print("\nColumn names:",tab.columns.tolist())
print()
```

```
Shape: (791, 8)

Column names: ['year', 'month', 'decimal date', 'average', 'deseasonalized',
'ndays', 'sdev', 'unc']
```

The dataset uses -9.99 (negative placeholders) for missing data ! I'll replace them with NaNs.

In [7]:
```
tab.replace(-9.99,np.nan,inplace=True)
tab.replace(-0.99,np.nan,inplace=True)
```

In [5]:
```
print(tab.describe())
```

```
              year       month  decimal date     average  deseasonalized  \
count   791.000000  791.000000    791.000000  791.000000      791.000000
mean   1990.624526    6.505689   1991.124379  358.783704      358.783198
std      19.041177    3.452705     19.041344   31.411617       31.362066
min    1958.000000    1.000000   1958.202700  312.430000      314.430000
25%    1974.000000    4.000000   1974.666650  330.230000      330.515000
50%    1991.000000    7.000000   1991.125000  354.930000      355.260000
75%    2007.000000    9.500000   2007.583350  384.280000      384.155000
max    2024.000000   12.000000   2024.041700  424.000000      422.560000

            ndays        sdev         unc
count  791.000000  791.000000  791.000000
mean    19.030341   -2.098521   -0.098647
std     12.009662    4.535248    0.512949
min     -1.000000   -9.990000   -0.990000
25%     10.500000    0.165000    0.065000
50%     25.000000    0.400000    0.150000
75%     28.000000    0.565000    0.210000
max     31.000000    1.310000    0.580000
```

In [8]:
```
# check for missing values
print("Missing values per column:\n", tab.isnull().sum())
print(f"\nTime span: {tab['year'].min()} - {tab['year'].max()}  "
      f"({tab['year'].max() - tab['year'].min()} years)")
print(f"Total months: {len(tab)}")
```

```
Missing values per column:
 year               0
month               0
decimal date        0
average             0
deseasonalized      0
ndays               0
sdev              196
unc               194
dtype: int64

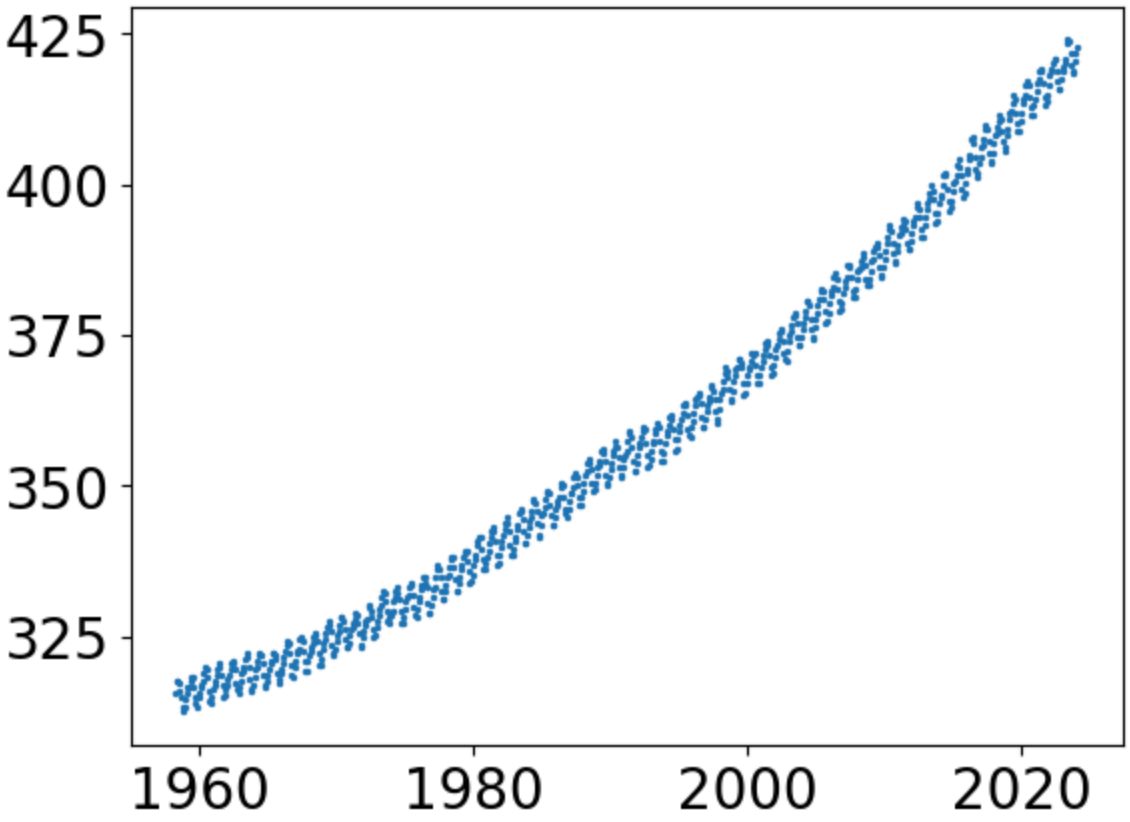
Time span: 1958 – 2024  (66 years)
Total months: 791
```

In [9]:
```
tab['decimal date']
```

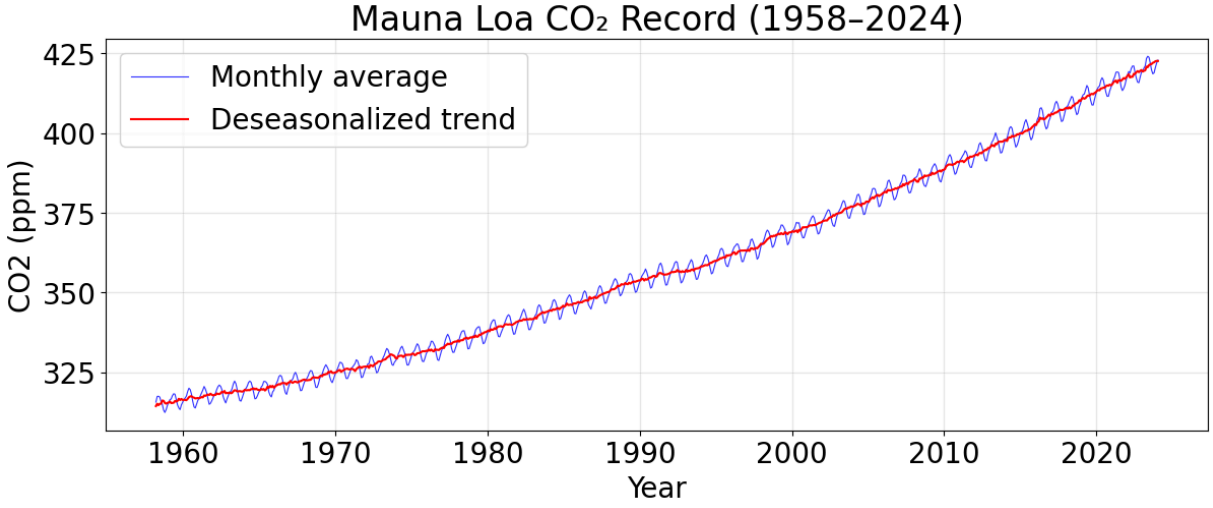
Out[9]:
```
0      1958.2027
1      1958.2877
2      1958.3699
3      1958.4548
4      1958.5370
         ...
786    2023.7083
787    2023.7917
788    2023.8750
789    2023.9583
790    2024.0417
Name: decimal date, Length: 791, dtype: float64
```

Time span is 1958 to 2024.

In [10]:
```
plt.scatter(tab['decimal date'],tab['average'], s=2)
plt.show()
```



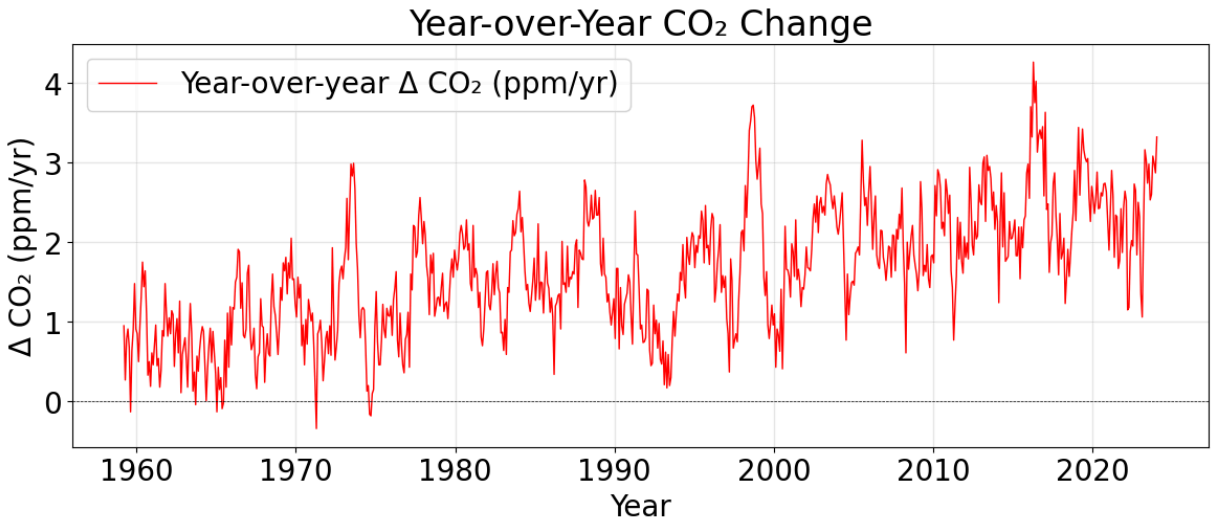
In [11]:
```
plt.figure(figsize=(14, 5))
plt.plot(tab['decimal date'], tab['average'], color='blue', lw=0.8,
alpha=0.8, label='Monthly average')
plt.plot(tab['decimal date'], tab['deseasonalized'], color='red', lw=1.5,
label='Deseasonalized trend')
plt.ylabel("CO2 (ppm)")
plt.xlabel("Year")
plt.title("Mauna Loa CO₂ Record (1958–2024)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



In [12]:
```
# Compute the year to year change ! For instance May 2020 CO2 minus May
2019 ,.. etc
```

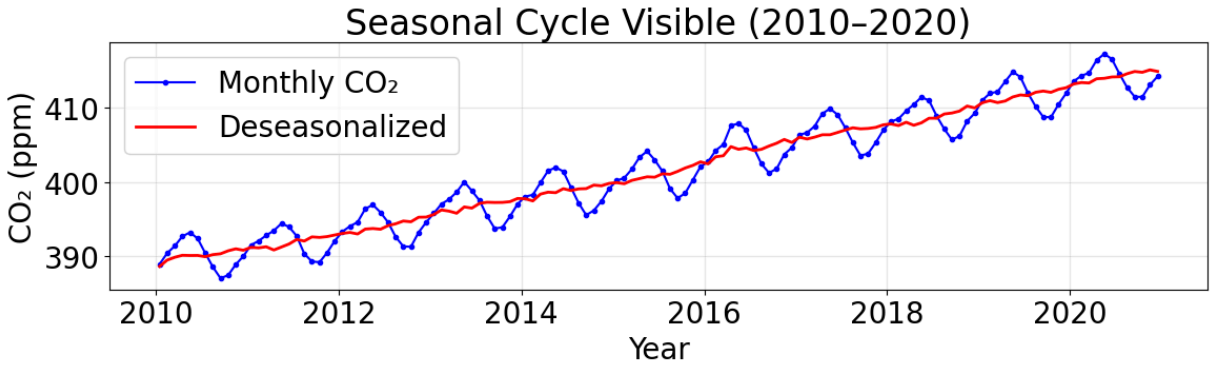
```
tab['yoy_change'] = tab['average'].diff(12)

plt.figure(figsize=(14, 5))
plt.plot(tab['decimal date'], tab['yoy_change'], color='red', lw=1.0,
label='Year-over-year Δ CO₂ (ppm/yr)')
plt.axhline(0, color='k', lw=0.5, linestyle='--')
plt.ylabel('Δ CO₂ (ppm/yr)')
plt.xlabel('Year')
plt.title('Year-over-Year CO₂ Change')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



In [13]:
```
#  Zoom in on a seasonal cycle, say the last decade !
recent = tab[(tab['year'] >= 2010) & (tab['year'] <= 2020)]

plt.figure(figsize=(12, 4))
plt.plot(recent['decimal date'], recent['average'],
         color='blue', lw=1.5, marker='o', ms=3, label='Monthly CO₂')
plt.plot(recent['decimal date'], recent['deseasonalized'],
         color='red', lw=2, label='Deseasonalized')
plt.xlabel('Year')
plt.ylabel('CO₂ (ppm)')
plt.title('Seasonal Cycle Visible (2010–2020)')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('co2_seasonal_zoom.png', dpi=150)
plt.show()
```



Part 4 Prepare Data

Question 2

(2 pts)

Are there outliers in the dataset? If so remove them and then create your feature (the decimal years) and target arrays (the average CO2 concentration). Use only the data from 1958 to 2010 for training. Make a separate array with the data from the beginning of 2011 to the end of 2019 for testing.

In [11]:
```
# code here
```

In [18]:
```
tab=tab.dropna(subset=['average'])
print(tab.shape)
```

```
(791, 9)
```

In [19]:
```
train = tab[tab['decimal date'] < 2011]
test  = tab[(tab['decimal date'] >= 2011) & (tab['decimal date'] < 2020)]
```

In [20]:
```
X_train = train['decimal date'].values
y_train = train['average'].values

X_test = test['decimal date'].values
y_test = test['average'].values
```

Part 5 Select model and train

In examining the model, we can see that there is a long term trend but also some periodic variations. We can use a combination of kernels to model the curve.

Some helpful references for using Gaussian Processes with Scikit Learn are here:

- https://scikit-learn.org/stable/modules/generated/sklearn.gaussian_process.GaussianProcessRegress
- https://scikit-learn.org/stable/modules/gaussian_process.html

Part 5.1 Practice with Kernels

To understand how to use Gaussian processes, we will practice with start by an example of using kernels in Scikit Learn.

We will start with the Radial Basis Kernel (also known as the squared exponential kernel). This kernel corresponds to a basis set of Gaussians. It is often used to fit time series data that has correlation over some length scale.

Start by reading about the RBF kernel in Scikit Learn: https://scikit-learn.org/stable/modules/generated/sklearn.gaussian_process.kernels.RBF.html

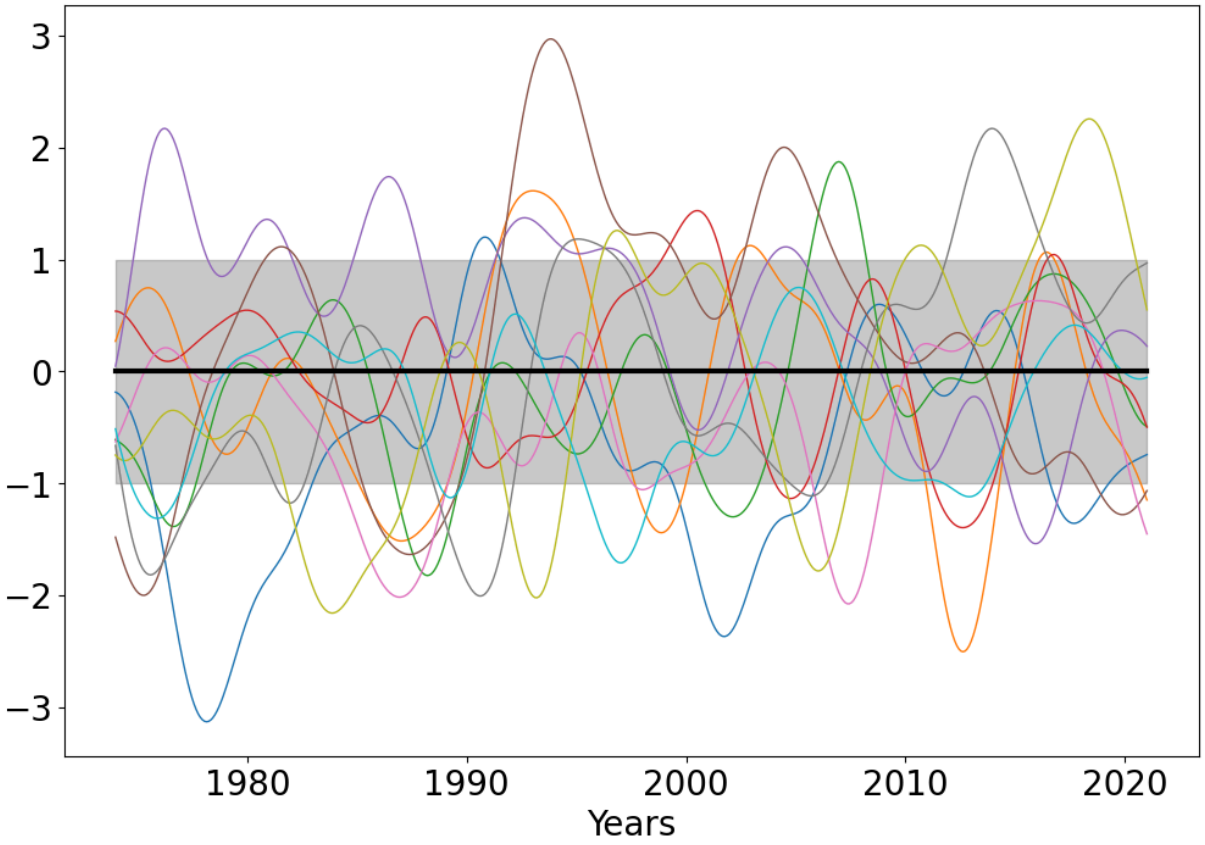
Below is some code used to initialize a Gaussian Process regression model and the RBF kernel. Before fitting the data, we can use this model to sample from the prior space of functions that are defined by the kernel. Often it is useful to see the shape and form of the functions that it can model. This will help you get a since of what kernels might be useful for modeling your data.

In [31]:
```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import (RBF, Matern,
RationalQuadratic,
                                              ExpSineSquared, DotProduct,
                                              ConstantKernel, WhiteKernel)
```

In [32]:
```
kernel = 1.0 * RBF(length_scale=2.0, length_scale_bounds=(1e-1, 10.0))

gp = GaussianProcessRegressor(kernel=kernel)
plt.figure(figsize=(12,8))
X_ = np.linspace(1974, 2021, 500)
y_mean, y_std = gp.predict(X_[:, np.newaxis], return_std=True)
plt.plot(X_, y_mean, 'k', lw=3, zorder=9)
plt.fill_between(X_, y_mean - y_std, y_mean + y_std,
                 alpha=0.2, color='k')
y_samples = gp.sample_y(X_[:, np.newaxis], 10)
plt.plot(X_, y_samples, lw=1)
plt.xlabel('Years')
```

Out[32]:
```
Text(0.5, 0, 'Years')
```



Question 3

(3 pts)

In the example code above, what are the hyperparameters of the `RBF` kernel? Describe the effect of those parameters.

The RBF kernel has two hyperparameters

- the length scale (set to 2 above). It controls how wiggly the sampled functions are. A large length scale means the function varies slowly and smoothlt over ong distances. A small lenght scale means the function can vary rapidly over short distances. For us, this would correspond to how quickly CO2 can change over time !
- the constant factor 1 above. This is the amplitude/ variance of the kernel and controls how far the sampled functions deviate from the mean. Larger values allow bigger up and down swings.

Question 4

(6 pts)

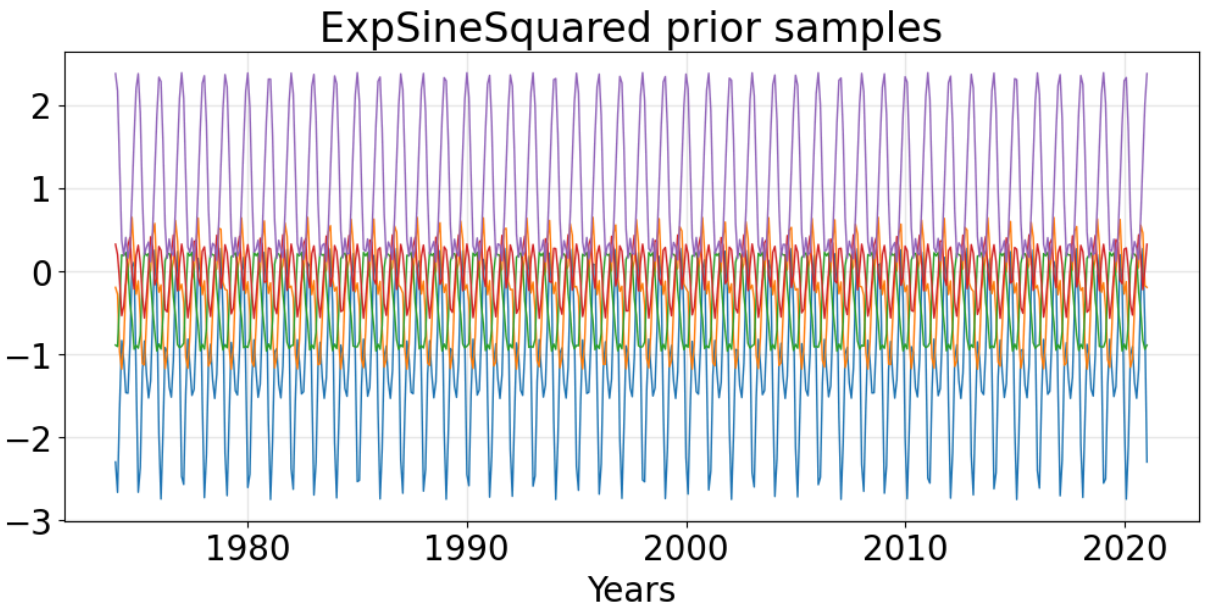
In a similar way as the example above, examine and plot some samples from the `ExpSineSquared` and `WhiteKernel`. What are the hyperparameters of these kernels? Describe qualitatitvely what the samples from the prior look like for these kernels.

In [ ]:
```
# code here
```

In [23]:
```
kernel = 1.0 * ExpSineSquared(length_scale=1.0, periodicity=1.0)
gp = GaussianProcessRegressor(kernel=kernel)

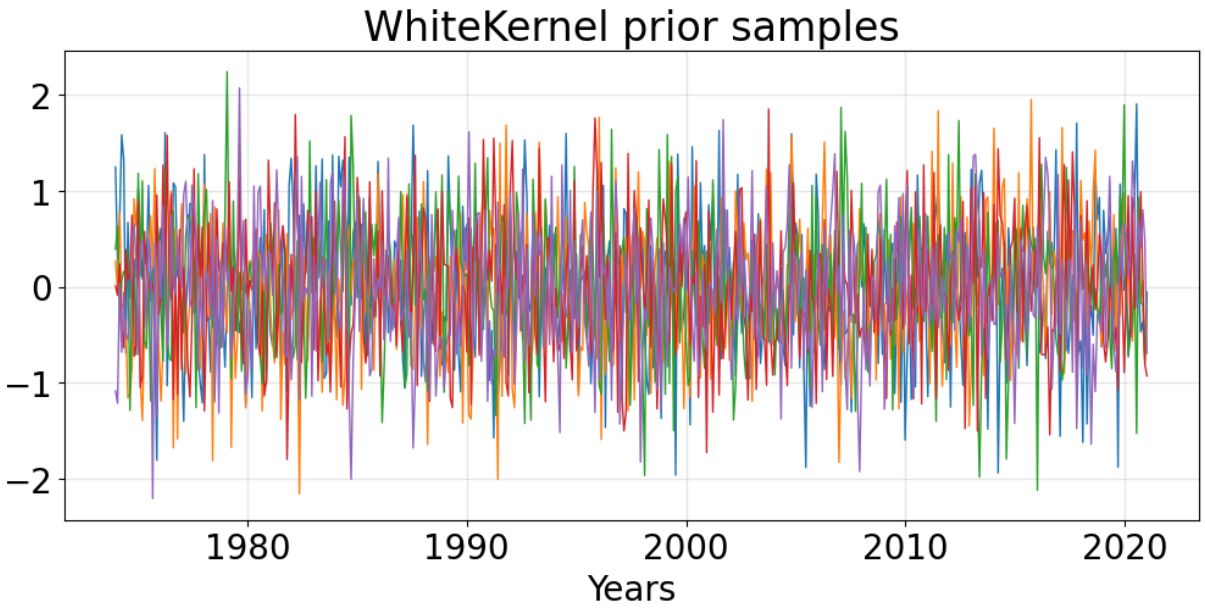
X_ = np.linspace(1974, 2021, 500)

plt.figure(figsize=(12, 5))
y_samples = gp.sample_y(X_[:, np.newaxis], 5)
plt.plot(X_, y_samples, lw=1)
plt.xlabel('Years')
plt.title('ExpSineSquared prior samples')
plt.grid(alpha=0.3)
plt.show()
```



In [29]:
```
kernel = WhiteKernel(noise_level=0.5)
gp = GaussianProcessRegressor(kernel=kernel)

plt.figure(figsize=(12, 5))
y_samples = gp.sample_y(X_[:, np.newaxis], 5)
plt.plot(X_, y_samples, lw=1)
plt.xlabel('Years')
plt.title('WhiteKernel prior samples')
plt.grid(alpha=0.3)
plt.show()
```



- Expsine squared has three parameters: the amplitude (1.0), the length scale (controls the smoothness within each period) and periodicity (controls the repeat interval). Samples look like smooth regular oscillatinve waves.
- whitekernel has only one hyperparameter -- the noise level ! Samples look like random noise with no structure or correlation between points - each point is independent ! It can be used to model measurement noise rather than any real signal.

Part 5.2 Put together a Gaussian Process Model

One of the great propertiers of Gaussian Processes is that you can simply add kernels together to make more complicated models.

We'll start off first with just the `RBF` kernel to fit the long term trends. Below is some example code to show how to do it with Scikit Learn.

In [35]:
```
k1 = 66.0**2 * RBF(length_scale=67.0)  # long term smooth rising trend

kernel_gpml = k1    # here we have just 1 kernel

gp = GaussianProcessRegressor(kernel=kernel_gpml,
alpha=0.01,normalize_y=True)

# NOTE: running the .fit method will fit for the hyper parameters of the
kernel. Fitting might take a while


## MODIFICATION - X TRAIN NEEDS TO BE RESHAPED TO 2D FOR SKLEARN

gp.fit(X_train.reshape(-1,1),y_train)
```

```
print("GPML kernel: %s" % gp.kernel_)
print("Log-marginal-likelihood: %.3f"
      % gp.log_marginal_likelihood(gp.kernel_.theta))
```

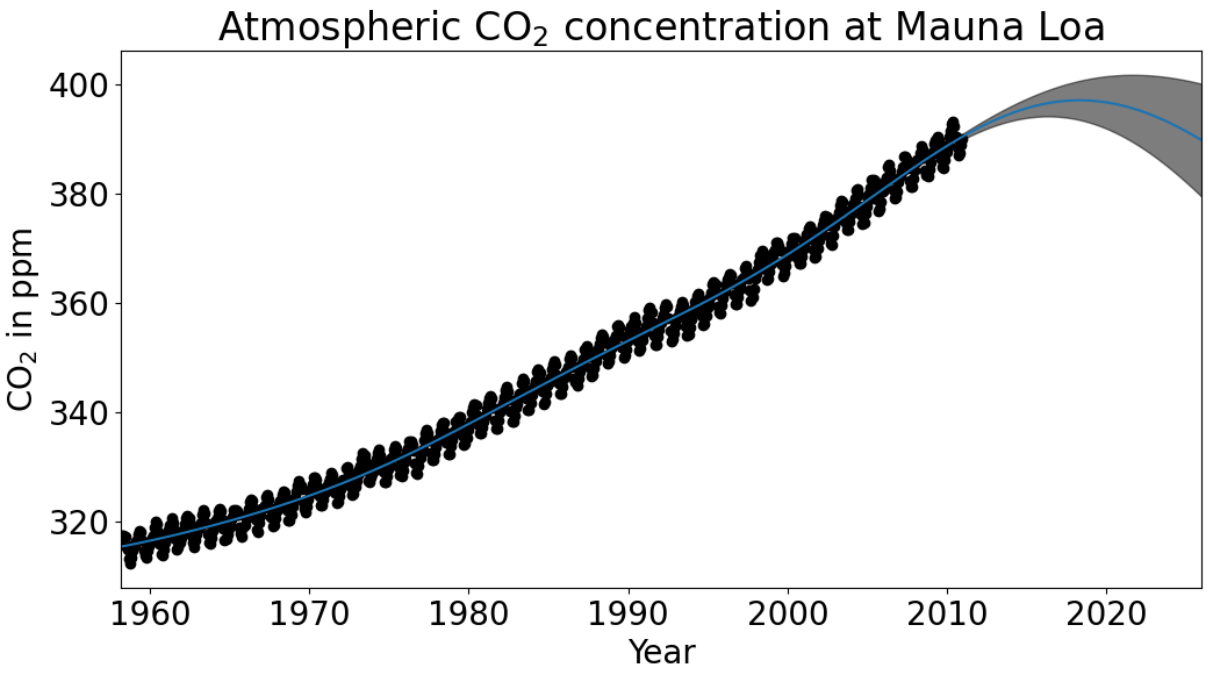
```
GPML kernel: 1.21**2 * RBF(length_scale=16.9)
Log-marginal-likelihood: 566.971
```

In [37]:
```
# plot the predictions

X_ = np.linspace(X_train.min(), X_train.max() + 15, 1000)[:, np.newaxis] #
predict 10 years into the future
y_pred, y_std = gp.predict(X_, return_std=True)

# Illustration
plt.figure(figsize=(12,6))
plt.scatter(X_train, y_train, c='k')
plt.plot(X_, y_pred)
plt.fill_between(X_[:, 0], y_pred - y_std, y_pred + y_std,
                 alpha=0.5, color='k')
plt.xlim(X_.min(), X_.max())
plt.xlabel("Year")
plt.ylabel(r"CO$_2$ in ppm")
plt.title(r"Atmospheric CO$_2$ concentration at Mauna Loa")
```

Out[37]: Text(0.5, 1.0, 'Atmospheric CO$_2$ concentration at Mauna Loa')



The RBF alone does a great job capturing the long term trend and even extrapolatese reasonably into the futrue. We note two clear issues !!

- COmpletely misses the seasonal wiggles and the prediction is too smppth
- The prediction curves back down after 2010. physically this is wrong and we do not expect CO2 to start decreasing.

Similarly, we can use instead the periodic kernel `ExpSineSquared` to model the periodic behavoir of the curve.

In [40]:
```
k2 = ExpSineSquared(length_scale=1.0, periodicity=1.0,
                    periodicity_bounds="fixed")
kernel_gpml = k2    # here we have just 1 kernel

gp = GaussianProcessRegressor(kernel=kernel_gpml,
alpha=0.01,normalize_y=True)

# NOTE: running the .fit method will fit for the hyper parameters of the
kernel. Fitting might take a while
gp.fit(X_train.reshape(-1,1),y_train)

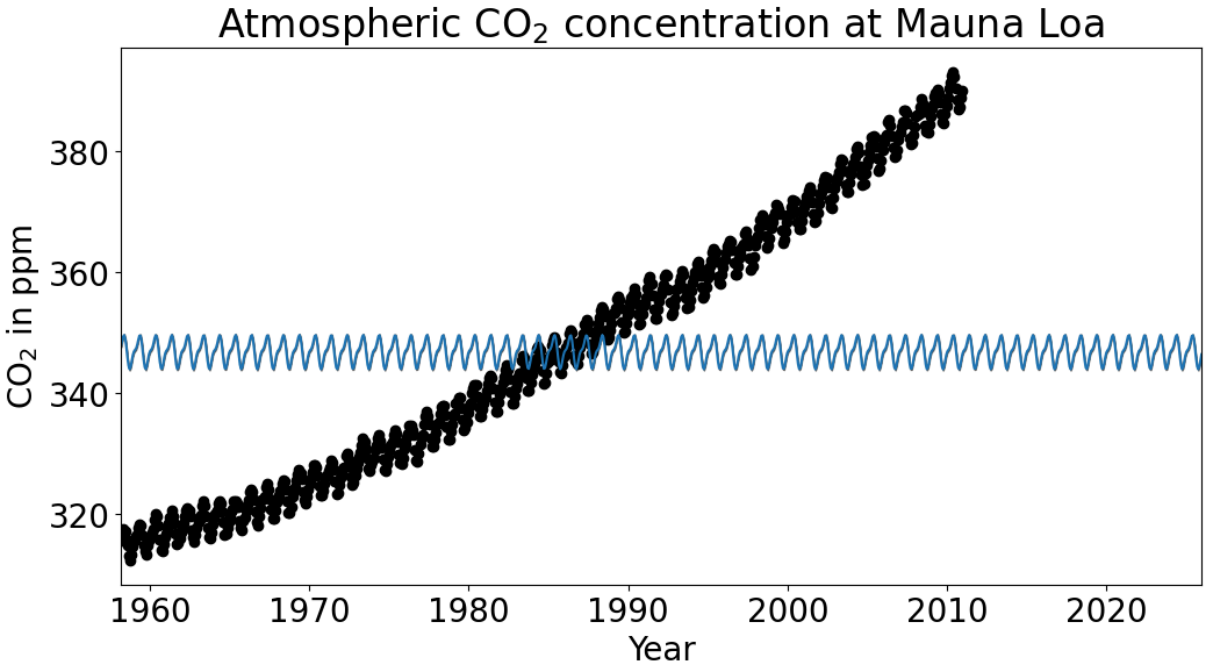
# plot the predictions

X_ = np.linspace(X_train.min(), X_train.max() + 15, 1000)[:, np.newaxis] #
predict 10 years into the future
y_pred, y_std = gp.predict(X_, return_std=True)

# Illustration
plt.figure(figsize=(12,6))
plt.scatter(X_train, y_train, c='k')
plt.plot(X_, y_pred)
plt.fill_between(X_[:, 0], y_pred - y_std, y_pred + y_std,
                 alpha=0.5, color='k')
plt.xlim(X_.min(), X_.max())
plt.xlabel("Year")
plt.ylabel(r"CO$_2$ in ppm")
plt.title(r"Atmospheric CO$_2$ concentration at Mauna Loa")

print("GPML kernel: %s" % gp.kernel_)
print("Log-marginal-likelihood: %.3f"
      % gp.log_marginal_likelihood(gp.kernel_.theta))
```

```
GPML kernel: ExpSineSquared(length_scale=4.43, periodicity=1)
Log-marginal-likelihood: -30637.170
```



This ix expected in the sense that ExpSineSquared alone captures the seasonal wiggle but is stuck as a aflat mean, completely ignoring the long term rising trend. The prediction flatlines into the future instead of continuing to rise !

Question 5

(4 pts)

Now create a new kernel by adding the `RBF` and the `ExpSineSquared` kernels together. Make a plot of the fit and the prediction. Plot the test data that you saved from 2011 to 2019. Describe the model performance.

In [49]:
```
# code here
k1 = 66.0**2 * RBF(length_scale=67.0)
k2 = ExpSineSquared(length_scale=1.0, periodicity=1.0,
periodicity_bounds="fixed")

kernel_gpml = k1 + k2
```

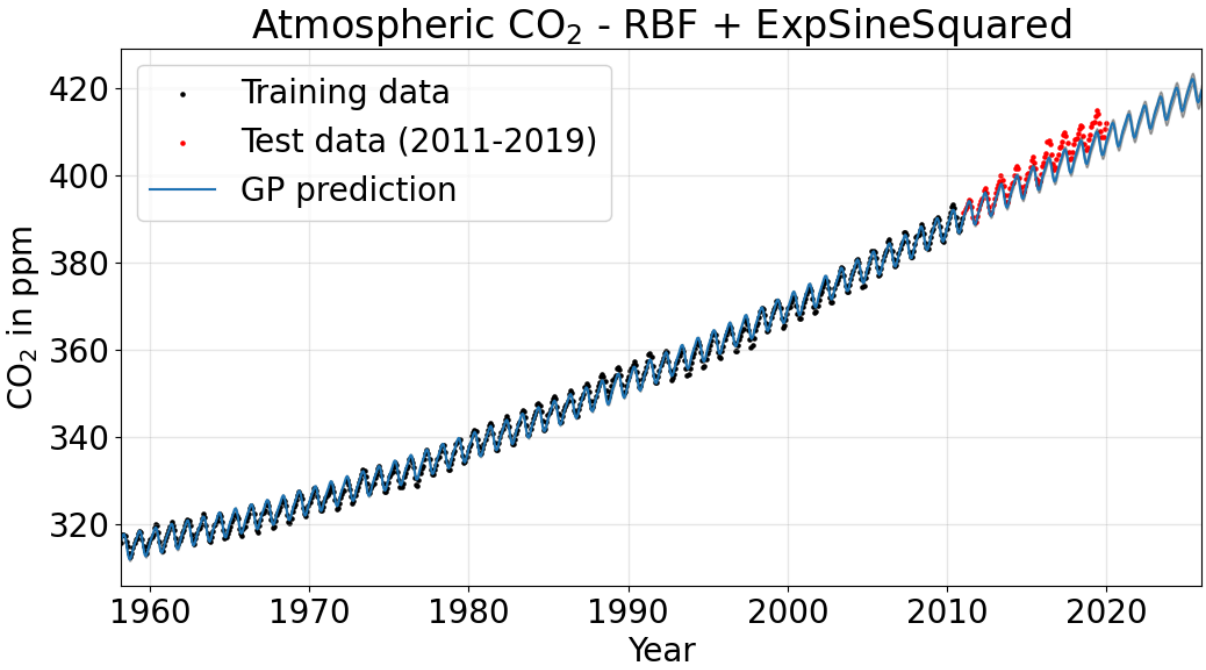
In [50]:
```
gp = GaussianProcessRegressor(kernel=kernel_gpml, alpha=0.01,
normalize_y=True)
gp.fit(X_train.reshape(-1, 1), y_train)

print("GPML kernel: %s" % gp.kernel_)
print("Log-marginal-likelihood: %.3f" %
gp.log_marginal_likelihood(gp.kernel_.theta))
```

```
GPML kernel: 6.15**2 * RBF(length_scale=108) + ExpSineSquared(length_scale=
5.09, periodicity=1)
Log-marginal-likelihood: 813.545
```

In [51]:
```
X_ = np.linspace(X_train.min(), X_train.max() + 15, 1000)[:, np.newaxis]
y_pred, y_std = gp.predict(X_, return_std=True)

plt.figure(figsize=(12, 6))
plt.scatter(X_train, y_train, c='k', s=5, label='Training data')
plt.scatter(X_test, y_test, c='red', s=5, label='Test data (2011-2019)')
plt.plot(X_, y_pred, label='GP prediction')
plt.fill_between(X_[:, 0], y_pred - y_std, y_pred + y_std, alpha=0.3,
color='k')
plt.xlim(X_.min(), X_.max())
plt.xlabel("Year")
plt.ylabel(r"CO$_2$ in ppm")
plt.title(r"Atmospheric CO$_2$ - RBF + ExpSineSquared")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



The combined kernel performs very well. Following few observations are made

- model captures both long term risign trend and seasonal wiggles
- the test data in red falls almost perfectly on the prediction line and the model generalizes well to unseen data
- the uncertainty band is tight even 15 years into the future suggesting the model is confident in its extrapolation
- the prediction continues rising linearly into the futture, whic his physically more reasonable that the RBF only model that curved back down

Part 6 Fine tune model

While the major properties of the CO2 curve was represented by our two kernel model, we can do better by allowing for some variations in the long term trend, variations in the periodic signal, and allowing for noise.

We can do this by adjusting and including more kernels.

Question 6

(6 pts)

Create a kernel the combination of four components:

- The same `RBF` kernel as in Part 5
- Modify the periodic `ExpSineSquared` kernel in Part 5 by **multipyling** it with another `RBF` kernel to allow the periodic signal to vary slightly.
- Add a `RationalQuadratic` kernel to account for medium term irregularies
- Add a `WhiteKernel` to account for white noise (Gaussian noise) in the modeling
- Hint: if you need help with this, see https://scikit-learn.org/stable/auto_examples/gaussian_process/plot_gpr_co2.html

Fit the data from 1958 to 2010, and the fit the GP model

In [52]:
```
# code here
k1 = 66.0**2 * RBF(length_scale=67.0)

k2 = 2.4**2 * RBF(length_scale=90.0) \
     * ExpSineSquared(length_scale=1.3, periodicity=1.0,
periodicity_bounds="fixed")

k3 = 0.66**2 * RationalQuadratic(length_scale=1.2, alpha=0.78)

k4 = 0.18**2 * RBF(length_scale=0.134) + WhiteKernel(noise_level=0.19**2)

kernel_gpml = k1 + k2 +k3 +k4
```

In [53]:
```
gp = GaussianProcessRegressor(kernel=kernel_gpml, alpha=0.01,
normalize_y=True)
gp.fit(X_train.reshape(-1, 1), y_train)

print("GPML kernel: %s" % gp.kernel_)
print("Log-marginal-likelihood: %.3f" %
gp.log_marginal_likelihood(gp.kernel_.theta))
```

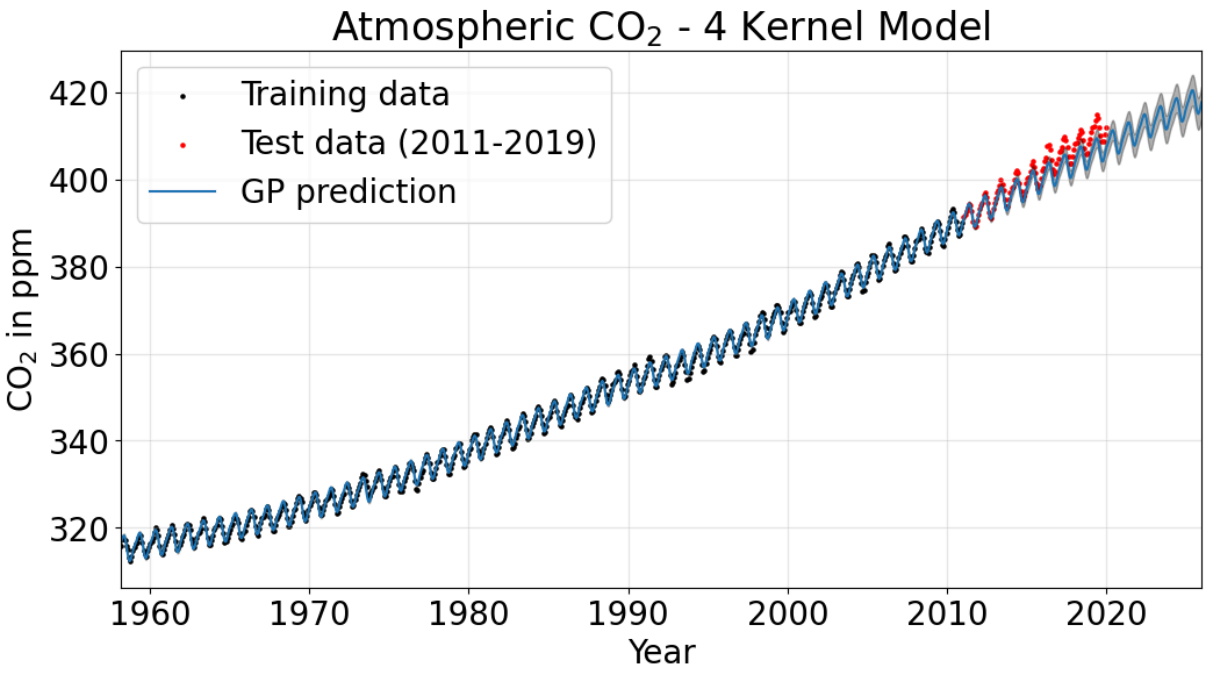
```
GPML kernel: 3.93**2 * RBF(length_scale=77.2) + 0.269**2 * RBF(length_scale=
3.2e+03) * ExpSineSquared(length_scale=2.53, periodicity=1) + 0.103**2 * Rat
ionalQuadratic(alpha=1.4, length_scale=123) + 0.0441**2 * RBF(length_scale=
6.92) + WhiteKernel(noise_level=1e-05)
Log-marginal-likelihood: 827.077
```

```
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/kernels.py:442: ConvergenceWarning: The optimal value found for dimen
sion 0 of parameter k2__k2__noise_level is close to the specified lower boun
d 1e-05. Decreasing the bound and calling fit again may find a better value.
  warnings.warn(
```

Plot the data and the predictions. Plot the 2011-2021 data and discuss how well it fits. Compare the prediction to that from Part 5.

In [54]:
```
# code here
X_ = np.linspace(X_train.min(), X_train.max() + 15, 1000)[:, np.newaxis]
y_pred, y_std = gp.predict(X_, return_std=True)

plt.figure(figsize=(12, 6))
plt.scatter(X_train, y_train, c='k', s=5, label='Training data')
plt.scatter(X_test, y_test, c='red', s=5, label='Test data (2011-2019)')
plt.plot(X_, y_pred, label='GP prediction')
plt.fill_between(X_[:, 0], y_pred - y_std, y_pred + y_std, alpha=0.3,
color='k')
plt.xlim(X_.min(), X_.max())
plt.xlabel("Year")
plt.ylabel(r"CO$_2$ in ppm")
plt.title(r"Atmospheric CO$_2$ - 4 Kernel Model")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



COmparison w the model in Part 5

- both models track the test data in red very well. Both models capture the long term wise and the seasonal wiggles well
- Uncertainty - the 4 kernel model has a wider uncertainty band because WhiteKenrl and RationalQuadratic explicitly model noise and irregularities, making the model

more honest about prediction uncertainty.
- the 2- kernel model predicts and overly smooth perfectly regular seasonal cycle, the 4-kernel model on the other hand allows for slight variations in amplitude fues to the RBF*ExpSineSquared term

The 4-kernel model is more realistic, it acknowledges that the data is noisy and that the seasonal cycle is not perfectly constant atthe cost of slightly wider uncertainty bands.

Question 7

(3 pts)

Explore some variations in the kernels to see if you can get a better prediction

In [59]:
```
# code here

# original is the original 4 kernel model
# larger lenght sclae where we try a larger lenght scale for a smoother
trend !
# Dot Product - where we add yet another kernel to capture the accelrating
trend !

kernels_to_try = {
    'original': k1 + k2 + k3 + k4,
    'larger_lengthscale': 66.0**2 * RBF(length_scale=100.0) + k2 + k3 + k4,
    'with_dotproduct': k1 + k2 + k3 + k4 + DotProduct(sigma_0=1.0),
}

for name, kernel in kernels_to_try.items():
    gp_try = GaussianProcessRegressor(kernel=kernel, alpha=0.01,
normalize_y=True)
    gp_try.fit(X_train.reshape(-1, 1), y_train)
    lml = gp_try.log_marginal_likelihood(gp_try.kernel_.theta)
    print(f"{name}: LML = {lml:.3f}")
```

```
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/kernels.py:442: ConvergenceWarning: The optimal value found for dimen
sion 0 of parameter k2__k2__noise_level is close to the specified lower boun
d 1e-05. Decreasing the bound and calling fit again may find a better value.
  warnings.warn(
original: LML = 827.077
```

```
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/kernels.py:442: ConvergenceWarning: The optimal value found for dimen
sion 0 of parameter k1__k2__k1__constant_value is close to the specified low
er bound 1e-05. Decreasing the bound and calling fit again may find a better
value.
  warnings.warn(
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/kernels.py:442: ConvergenceWarning: The optimal value found for dimen
sion 0 of parameter k2__k1__k1__constant_value is close to the specified low
er bound 1e-05. Decreasing the bound and calling fit again may find a better
value.
  warnings.warn(
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/kernels.py:442: ConvergenceWarning: The optimal value found for dimen
sion 0 of parameter k2__k2__noise_level is close to the specified lower boun
d 1e-05. Decreasing the bound and calling fit again may find a better value.
  warnings.warn(
larger_lengthscale: LML = 566.939
with_dotproduct: LML = 552.802
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/_gpr.py:660: ConvergenceWarning: lbfgs failed to converge (status=2):
ABNORMAL_TERMINATION_IN_LNSRCH.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
  _check_optimize_result("lbfgs", opt_res)
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/kernels.py:442: ConvergenceWarning: The optimal value found for dimen
sion 0 of parameter k1__k2__k2__noise_level is close to the specified lower
bound 1e-05. Decreasing the bound and calling fit again may find a better va
lue.
  warnings.warn(
```

After trying variations including a larger RBF length scale and adding a DotProduct kernel, the original 4-kernel combination achieved the highest log marginal likelihood. Increasing the length scale of the long term RBF made the trend too smooth, while adding a DotProduct kernel did not improve the fit. This suggests the original kernel hyperparameters were already near optimal for this dataset.

Assessing the effects of COVID19 on CO2 levels

Question 8

(2 pts)

First, without looking at the data, predict would happen to the CO2 levels in 2020 to recent times. Could the COVID19 pandemic affect these levels? Why or why not?

During the COVID-19 pandemic (2020–2021), global economic activity dropped sharply eg. factories shut down, air travel nearly stopped, and commuting collapsed. Since

burning fossil fuels is the primary driver of rising atmospheric CO2, one might expect the rate of increase to slow down during this period. However, CO2 levels in the atmosphere are unlikely to decrease outright, i.e they would just rise more slowly. This is because even a large drop in human emissions is small relative to the total amount of CO2 already in the atmosphere, and natural sources/sinks (oceans, forests) continue operating regardless. So the prediction would be: CO2 continues rising through 2020–2021, but perhaps at a slightly slower rate than the pre-pandemic trend.

Question 9

(2 pts)

Now, go back and fit your GP model on data from up to 2019. Then, predict the CO2 levels in 2020 to the most recent year

In [56]:
```
# fit on data up to end of 2019
train_q9 = tab[tab['decimal date'] < 2020]

X_train_q9 = train_q9['decimal date'].values.reshape(-1, 1)
y_train_q9 = train_q9['average'].values

gp_q9 = GaussianProcessRegressor(kernel=kernel_gpml, alpha=0.01,
normalize_y=True)
gp_q9.fit(X_train_q9, y_train_q9)
```

```
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/gaussian_p
rocess/kernels.py:442: ConvergenceWarning: The optimal value found for dimen
sion 0 of parameter k2__k2__noise_level is close to the specified lower boun
d 1e-05. Decreasing the bound and calling fit again may find a better value.
  warnings.warn(
```

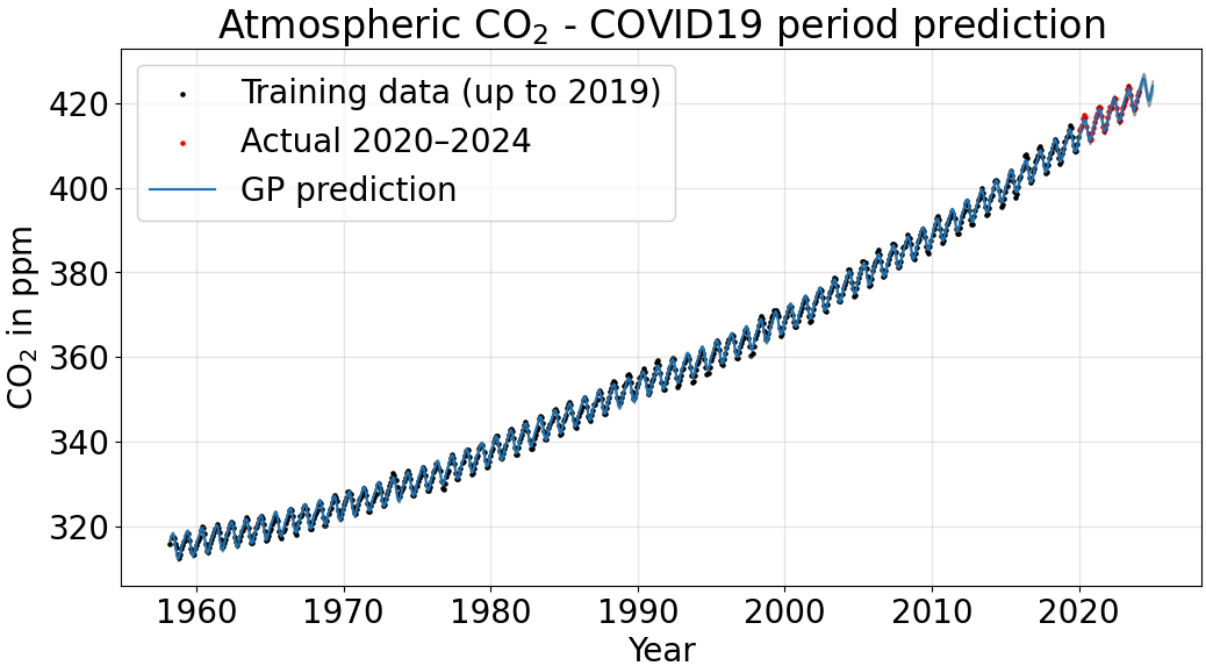
Out[56]:
```
GaussianProcessRegressor(alpha=0.01,
                         kernel=66**2 * RBF(length_scale=67) + 2.4*
*2 * RBF(length_scale=90) * ExpSineSquared(length_scale=1.3, period
icity=1) + 0.66**2 * RationalQuadratic(alpha=0.78, length_scale=1.
2) + 0.18**2 * RBF(length_scale=0.134) + WhiteKernel(noise_level=0.
0361),
                         normalize_y=True)
```

In [58]:
```
# predict up to most recent year in the dataset (2024)
X_ = np.linspace(X_train_q9.min(), 2025, 1000)[:, np.newaxis]
y_pred, y_std = gp_q9.predict(X_, return_std=True)

# get actual post-2020 data to compare
actual = tab[tab['decimal date'] >= 2020]

plt.figure(figsize=(12, 6))
```

```
plt.scatter(X_train_q9, y_train_q9, c='k', s=5, label='Training data (up to
2019)')
plt.scatter(actual['decimal date'], actual['average'], c='red', s=5,
label='Actual 2020–2024')
plt.plot(X_[:, 0], y_pred, label='GP prediction')
plt.fill_between(X_[:, 0], y_pred - y_std, y_pred + y_std, alpha=0.3,
color='k')
plt.xlabel("Year")
plt.ylabel(r"CO$_2$ in ppm")
plt.title(r"Atmospheric CO$_2$ – COVID19 period prediction")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Question 10

(3 pts)

Describe whether what you found meets your expectations. Why or why not?

- The result largely matches our expectations from Q8. We predicted that CO2 would continue rising through 2020–2024, just possibly at a slower rate - and indeed the actual data shows continuous rising with no dip or pause.
- However, the result is perhaps more surprising in its precision — the actual data doesn't even show a detectable slowdown in the rate of increase. The red points fall almost perfectly on the prediction line, meaning the pandemic had essentially zero measurable impact on atmospheric CO2 levels.

This makes physical sense: global CO2 emissions dropped by roughly 7% in 2020, but this is a small perturbation on top of the ~3 trillion tonnes of CO2 already in the atmosphere. The ocean and land carbon sinks also adjust, further dampening any signal. The conclusion is sobering !! Even a once-in-a-century global shutdown is not enough to meaningfully alter the CO2 trajectory. Structural, long-term changes to energy systems would be required to show up in this data.

Part 7 Present solution

Question 11

(4 pts)

Discuss the model which appears to perform best: what kernels did you end up using? What are the hyperparameters of the best kernel?

The best performing model was the original 4 kernel model from Q6, which achieved the highest log marginal likelihood compared to the variations tried in Q7. The kernel combination was:

- k1 RBF - captures the long term smooth rising trend with an optimized length of about 67 years
- k2 RBF*ExpSIneSquared - captures the seasonal cycle with periodicity fixed at 1 year, multiplied by an RBF to allow the amplitude to vary slowly over time
- k3 RationalQuadratice - captures medium term irregularities that do not fit neatly into the trend or seasonal components
- k4 RBF + WhiteKernel - accounts for short term noise and measurement uncertainty

Part 8 Launch, Monitor and Maintain

Nothing to here for this lab.

All done! Nice job!